

Partager l'info c'est progresser ensemble



DELPHI, PROGRAMMATION ORIENTEE OBJET

5 Thématiques

- Introduction à la POO
- Classes et objets
- Héritage
- Polymorphisme
- Encapsulation

Niveau débutant

22 exemples et un cas pratique
« gestion des documents »

TABLE DES MATIERES

1. ORGANISATION DU TUTORIAL	3
2. INTRODUCTION A LA POO	4
3. PRESENTATION DE DELPHI	5
4. CREATION DE CLASSES	7
5. HERITAGE	9
6. POLYMORPHIE	11
7. HERITAGE MULTIPLE	14
8. INTERFACES	16
9. CAS PRATIQUE « GESTION DES DOCUMENTS »	19

LISTE DES EXEMPLES

Exemple 1 : Déclaration et implémentation d'une classe	7
Exemple 2 : Utilisation de la classe dans un pro	8
Exemple 3 : Explication de la notion de l'héritage (Classe de base)	9
Exemple 4 : Explication de la notion de l'héritage (Classe dérivée)	10
Exemple 5 : Explication de la notion de l'héritage (Exécution)	11
Exemple 6 : utilisation de la polymorphie	11
Exemple 7 : Dériver et polymorphie (Première classe dérivée)	12
Exemple 8 : Dériver et polymorphie (Deuxième classe dérivée)	12
Exemple 9 : Dériver et polymorphie (Utilisation)	13
Exemple 10 : Héritage multiple (Première classe parente)	14
Exemple 11 : Héritage multiple (Deuxième classe parente)	15
Exemple 12 : Héritage multiple (Dérivation des classes précédentes)	15
Exemple 13 : Héritage multiple (Utilisation)	16
Exemple 14 : Interface	17
Exemple 15 : Classe implémentant l'interface	17
Exemple 16 : Utilisation de la classe implémentant l'interface	18
Exemple 17 : La classe TDocument	20
Exemple 18 : La classe TBook	21
Exemple 19 : Utilisation de la classe TBook	23
Exemple 20 : Création de la classe TReport	23
Exemple 21 : Implémentation de la classe TReport	24
Exemple 22 : Utilisation de la classe TReport	24

1. Organisation du tutorial

Pour faciliter la lecture de ce tutorial, il a été organisé comme suit :

- **Introduction à la POO** : expliquer les concepts clés de la POO tels que l'encapsulation, l'héritage et le polymorphisme, ainsi que les avantages de la POO par rapport aux autres approches de programmation.
- **Présentation de Delphi** : donner une vue d'ensemble de Delphi, son environnement de développement intégré (IDE), ses fonctionnalités et les langages de programmation supportés.
- **Classes et objets** : expliquer comment définir et créer des classes et des objets en Delphi, et montrer comment utiliser les propriétés et les méthodes d'un objet.
- **Héritage** : montrer comment utiliser l'héritage en Delphi pour créer des classes dérivées à partir de classes existantes, en expliquant les concepts de base de l'héritage et les avantages de cette approche.
- **Polymorphisme** : expliquer comment utiliser le polymorphisme en Delphi pour créer des méthodes virtuelles et pour appeler des méthodes d'objets dérivés via une référence à la classe de base.
- **Encapsulation** : montrer comment utiliser l'encapsulation en Delphi pour protéger les données et les méthodes d'un objet en les rendant privées ou protégées, et comment accéder à ces données et méthodes à travers des méthodes publiques.
- **Interfaces** : expliquer comment utiliser les interfaces en Delphi pour implémenter des fonctionnalités communes à plusieurs classes, en utilisant des méthodes et des propriétés partagées.
- **Exemples pratiques** : présenter des exemples pratiques de l'utilisation de la POO en Delphi pour résoudre des problèmes courants, tels que la création d'une application de gestion de contacts ou d'une application de jeu simple.
- **Conclusion** : résumer les concepts clés de la POO en Delphi et les avantages de cette approche, ainsi que fournir des ressources supplémentaires pour ceux qui souhaitent en savoir plus sur le sujet.

En suivant ce plan de travail progressif, vous pouvez aider les apprenants à comprendre les concepts de la POO en Delphi de manière efficace et à appliquer ces concepts dans des projets pratiques.

2. Introduction à la POO

La programmation orientée objet (POO) est une approche de programmation qui repose sur la création d'objets interactifs pour représenter les éléments d'un programme. Les objets sont des structures de données qui contiennent des propriétés (variables) et des méthodes (fonctions) qui permettent de les manipuler.

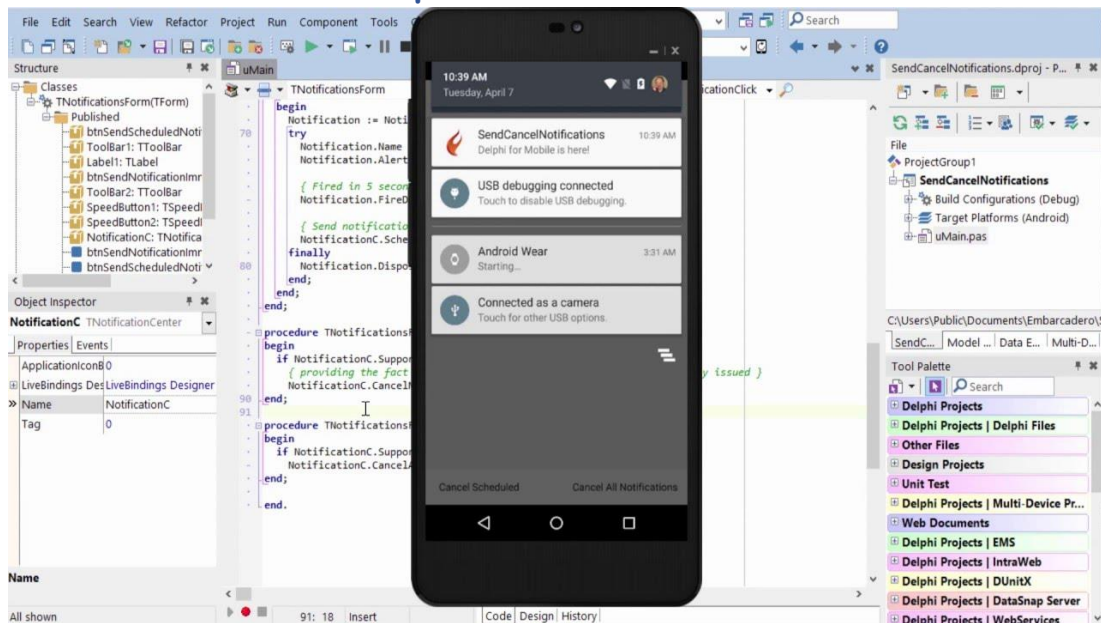
Les concepts clés de la POO incluent l'encapsulation, l'héritage et le polymorphisme. L'encapsulation consiste à protéger les données et les méthodes d'un objet en les rendant privées ou protégées, et à les rendre accessibles uniquement via des méthodes publiques. L'héritage permet de créer des classes dérivées à partir de classes existantes, en héritant des propriétés et des méthodes de la classe de base. Le polymorphisme permet de créer des méthodes virtuelles qui peuvent être redéfinies dans des classes dérivées, ce qui permet d'appeler des méthodes d'objets dérivés via une référence à la classe de base.

Les avantages de la POO sont nombreux :

- Une meilleure organisation et structuration du code ;
- Une réutilisation du code plus facile ;
- Une maintenance plus simple du code ;
- Une abstraction plus efficace des données ;
- Une meilleure collaboration entre les membres de l'équipe de développement.

Dans Delphi, la POO est mise en œuvre via le langage de programmation *Object Pascal*, qui prend en charge toutes les fonctionnalités de la POO. Dans les sections suivantes, nous verrons comment utiliser la POO en Delphi pour créer des classes, des objets, des interfaces, etc.

3. Présentation de Delphi



Delphi est un environnement de développement intégré (IDE) utilisé pour le développement de logiciels pour Windows, macOS, Android et iOS. Delphi prend en charge plusieurs langages de programmation, notamment Object Pascal, C++, C#, et JavaScript.

Delphi propose une variété de fonctionnalités pour faciliter le développement de logiciels, telles que des outils de conception visuelle, des éditeurs de code, des débogueurs et des outils de profilage. Les développeurs peuvent également utiliser les composants de la bibliothèque de composants VCL (Visual Component Library) pour créer des interfaces utilisateur et des fonctionnalités communes dans leurs applications.

Delphi prend en charge la POO via le langage de programmation Object Pascal, qui a été développé par Borland en 1986, il s'agit plus particulièrement de l'encapsulation, l'héritage et le polymorphisme.

Dans Delphi, les développeurs peuvent utiliser l'IDE pour créer des projets, des unités de code et des formulaires. Les projets peuvent être compilés en exécutables ou en bibliothèques de liens dynamiques (DLL), et les formulaires peuvent être conçus en utilisant l'éditeur de conception visuelle de Delphi.

Dans les sections suivantes, nous verrons comment utiliser Delphi pour créer des classes et des objets en utilisant le langage Object Pascal.

Show and hide non-visual controls on your form

Identify components in the Structure pane through their corresponding icons

Filter properties and events using the built-in search functionality

Code Navigation Toolbar for easy access to classes and methods

Project Statistics to get a clear picture of team productivity

Clipboard History for access to previously copied content

Multi-Paste for modifying previously copied text and pasting it into the Code Editor

The screenshot shows the Delphi IDE interface. The main window displays a code editor with a Pascal program. The left sidebar contains the Structure pane, Object Inspector, and Properties/Events tabs. The top menu bar includes File, Edit, Search, View, Refactor, Project, Run, Component, Tools, Window, and Help. The bottom status bar shows the current line and column (31, 28) and the active window (Code | Design | History |).

The Structure pane shows the following components:

- Form1
 - Layout1
 - Label1
 - Button1
 - WebBrowser1
 - LocationSensor1
 - MultiView1
 - ListBox1
 - ToolBar2
 - Button2

The Object Inspector shows the following properties and events for MultiView1:

- TabOrder: 4
- Tag: 0
- TargetControl: Layout1
- TouchTargetExp: (TSounds)

The Project Statistics window shows the following data:

Category	Time
Editing	0:01:58
Designing	0:00:01
Inspecting	0:00:01
Compiling	0:00:06
Other	0:02:37
Total	0:04:42

The Clipboard History window shows the following content:

- Copy to Clipboard
- Insert in Editor
- AppAnalytics1.AllowTracking := Reader.ReadBoolean;
- Object ClearSaveStateButton: TButton
- Anchors = (akTop)
- Margins.Left = 15.000000000000000000
- Margins.Top = 15.000000000000000000
- Margins.Right = 15.000000000000000000
- if not FClearState and AppAnalytics1.AllowTracking
- AppAnalytics1.UserID;
- AppAnalytics1.Free;

The Multi-Paste window shows the following options:

- Paste Options
- Text before each line: DM1.FDQuery1.SQL.Add(
- Text after each line:);
- Escape Quotes: ☒
- Pascal style: ' = "
- Trim Clipboard Contents: ☒
- Preview: DM1.FDQuery1.SQL.Add(select invoice.id, invo
- DM1.FDQuery1.SQL.Add(from invoice, customer
- DM1.FDQuery1.SQL.Add(where invoice.customer=
- DM1.FDQuery1.SQL.Add(and customer.country=

4. Création de classes

Une classe est un type de données qui contient des propriétés (variables) et des méthodes (fonctions) permettant de manipuler les données. Pour créer une classe en Delphi, vous devez définir une nouvelle unité de code et y définir la classe.

Voici un exemple simple de classe en Delphi :

Exemple 1 : Déclaration et implémentation d'une classe

```
unit MaClasse;  
interface  
type  
    TMaClasse = class  
    private  
        FValeur: Integer;  
    public  
        constructor Create;  
        function GetValeur: Integer;  
        procedure SetValeur(Value: Integer);  
    end;  
  
implementation  
  
constructor TMaClasse.Create;  
begin  
    FValeur := 0;  
end;  
  
function TMaClasse.GetValeur: Integer;  
begin  
    Result := FValeur;  
end;  
  
procedure TMaClasse.SetValeur(Value: Integer);  
begin  
    FValeur := Value;  
end;  
  
end.
```

Dans cet exemple, nous avons défini une nouvelle unité de code nommée "MaClasse". Nous avons également défini une nouvelle classe appelée "TMaClasse", qui contient une propriété privée appelée "FValeur" et trois méthodes publiques : "Create", "GetValeur" et "SetValeur".

La méthode "Create" est un constructeur qui est appelé lorsqu'un nouvel objet est créé. La méthode "GetValeur" retourne la valeur de la propriété "FValeur", et la méthode "SetValeur" modifie la valeur de la propriété "FValeur".

Très important

En programmation Delphi, un constructeur (ou "constructor" en anglais) est une méthode spéciale d'une classe qui est appelée automatiquement lorsqu'un objet de cette classe est créé. Le constructeur est utilisé pour initialiser les propriétés et les champs de l'objet afin de s'assurer que l'objet est correctement configuré et prêt à être utilisé.

Le constructeur est souvent utilisé pour assigner des valeurs par défaut aux propriétés de l'objet, ou pour effectuer des opérations d'initialisation complexes, telles que l'ouverture de fichiers, la connexion à des bases de données ou la configuration d'autres objets nécessaires à l'objet principal.

En Delphi, le nom du constructeur est toujours le même que celui de la classe, et il n'a pas de type de retour explicite. Par exemple, si vous avez une classe "TMyClass", son constructeur s'appellera "constructor TMyClass". Vous pouvez avoir plusieurs constructeurs pour une même classe, avec des paramètres différents pour chaque constructeur, ce qui permet de créer des objets de différentes manières en fonction des besoins spécifiques de votre application.

Pour utiliser cette classe dans un autre code Delphi, vous devez ajouter l'unité "MaClasse" à votre projet et créer une instance de la classe "TMaClasse" comme ceci :

Exemple 2 : Utilisation de la classe dans un pro

```
program MonProgramme;

{$APPTYPE CONSOLE}

uses
  SysUtils, MaClasse in 'MaClasse.pas';

var
  MaVariable: TMaClasse;
begin
  MaVariable := TMaClasse.Create;
  MaVariable.SetValeur(42);
  Writeln(MaVariable.GetValeur);
  Readln;
end.
```

Dans cet exemple, nous avons créé une nouvelle instance de la classe "TMaClasse" en appelant la méthode "Create". Nous avons ensuite utilisé la méthode "SetValeur" pour modifier la valeur de la propriété "FValeur" de l'objet, et la méthode "GetValeur" pour obtenir la valeur de la propriété "FValeur". Le résultat sera l'affichage de "42" dans la console.

5. Héritage

L'héritage est une technique qui permet de créer de nouvelles classes en se basant sur une classe existante. La nouvelle classe hérite des propriétés et des méthodes de la classe existante, et peut également ajouter de nouvelles propriétés et méthodes.

Pour illustrer l'utilisation de l'héritage en Delphi, voici un exemple de classe qui hérite d'une classe de base :

Exemple 3 : Explication de la notion de l'héritage (Classe de base)

```
unit MaClasseDeBase;
interface

type
  TClasseDeBase = class
  private
    FValeur: Integer;
  public
    constructor Create;
    function GetValeur: Integer;
    procedure SetValeur(Value: Integer);
  end;

implementation

constructor TClasseDeBase.Create;
begin
  FValeur := 0;
end;

function TClasseDeBase.GetValeur: Integer;
begin
  Result := FValeur;
end;

procedure TClasseDeBase.SetValeur(Value: Integer);
begin
  FValeur := Value;
end;

end.
```

Dans cet exemple, nous avons défini une classe de base appelée "TClasseDeBase" qui contient une propriété privée appelée "FValeur" et trois méthodes publiques : "Create", "GetValeur" et "SetValeur".

Maintenant, nous allons créer une nouvelle classe qui hérite de la classe de base :

Exemple 4 : Explication de la notion de l'héritage (Classe dérivée)

```
unit MaClasseDerivee;  
  
interface  
  
uses  
    MaClasseDeBase;  
  
type  
    TClasseDerivee = class(TClasseDeBase)  
    public  
        function Multiplier(Valeur: Integer): Integer;  
    end;  
  
implementation  
  
function TClasseDerivee.Multiplier(Valeur: Integer): Integer;  
begin  
    Result := GetValeur * Valeur;  
end;  
  
end.
```

Dans cet exemple, nous avons défini une nouvelle unité de code appelée "MaClasseDerivee". Nous avons également utilisé l'instruction "uses" pour inclure l'unité "MaClasseDeBase" qui contient la classe de base.

Nous avons ensuite défini une nouvelle classe appelée "TClasseDerivee" qui hérite de la classe de base en utilisant l'instruction "class(TClasseDeBase)". La nouvelle classe contient une méthode publique appelée "Multiplier" qui multiplie la valeur stockée dans la propriété "FValeur" par une valeur passée en paramètre.

Pour utiliser cette classe dans un autre code Delphi, vous devez ajouter les unités "MaClasseDeBase" et "MaClasseDerivee" à votre projet et créer une instance de la classe "TClasseDerivee". Par exemple :

Exemple 5 : Explication de la notion de l'héritage (Exécution)

```
program MonProgramme;

{$APPTYPE CONSOLE}

uses
  SysUtils, MaClasseDeBase in 'MaClasseDeBase.pas',
  MaClasseDerivee in 'MaClasseDerivee.pas';

var
  MaVariable: TClasseDerivee;
begin
  MaVariable := TClasseDerivee.Create;
  MaVariable.SetValeur(10);
  Writeln(MaVariable.Multiplier(5));
  Readln;
end.
```

Dans cet exemple, nous avons créé une nouvelle instance de la classe "TClasseDerivee" en appelant la méthode "Create". Nous avons ensuite utilisé la méthode "SetValeur" pour modifier la valeur de la propriété "FValeur" de l'objet, puis nous avons appelé la méthode Multiplier de la classe Dérivée.

6. Polymorphie

La polymorphie est une technique qui permet de traiter des objets de différentes classes de manière homogène. Cela signifie que vous pouvez utiliser une méthode avec le même nom et les mêmes paramètres pour différents types d'objets, et le compilateur choisira automatiquement la méthode appropriée en fonction du type d'objet que vous utilisez.

Voici un exemple pour illustrer l'utilisation de la polymorphie en Delphi :

Exemple 6 : utilisation de la polymorphie

```
unit MaClasseDeBase;

interface

type
  TClasseDeBase = class
  public
    procedure AfficherMessage; virtual;
  end;

implementation

procedure TClasseDeBase.AfficherMessage;
begin
  Writeln('Je suis une instance de TClasseDeBase');
end;

end.
```

Dans cet exemple, nous avons défini une classe de base appelée "TClasseDeBase" qui contient une méthode publique appelée "AfficherMessage" avec le mot clé "virtual". Le mot clé "virtual" indique que cette méthode peut être redéfinie dans des classes dérivées.

Maintenant, nous allons créer deux classes dérivées qui redéfinissent la méthode "AfficherMessage" :

Exemple 7 : Dériver et polymorphie (Première classe dérivée)

```
unit MaClasseDerivee1;

interface

uses
    MaClasseDeBase;

type
    TClasseDerivee1 = class(TClasseDeBase)
    public
        procedure AfficherMessage; override;
    end;

implementation

procedure TClasseDerivee1.AfficherMessage;
begin
    Writeln('Je suis une instance de TClasseDerivee1');
end;

end.
```

Exemple 8 : Dériver et polymorphie (Deuxième classe dérivée)

```
unit MaClasseDerivee2;

interface

uses
    MaClasseDeBase;

type
    TClasseDerivee2 = class(TClasseDeBase)
    public
        procedure AfficherMessage; override;
    end;

implementation

procedure TClasseDerivee2.AfficherMessage;
begin
    Writeln('Je suis une instance de TClasseDerivee2');
end;

end.
```

Dans ces exemples, nous avons défini deux classes dérivées appelées "TClasseDerivee1" et "TClasseDerivee2" qui héritent de la classe de base "TClasseDeBase". Ces deux classes redéfinissent la méthode "AfficherMessage" en utilisant le mot clé "override" pour indiquer qu'elles remplacent la méthode de la classe de base.

Maintenant, nous allons utiliser ces classes dans notre code principal :

Exemple 9 : Dériver et polymorphie (Utilisation)

```
program MonProgramme;

{$APPTYPE CONSOLE}

uses
  SysUtils, MaClasseDeBase in 'MaClasseDeBase.pas',
  MaClasseDerivee1 in 'MaClasseDerivee1.pas',
  MaClasseDerivee2 in 'MaClasseDerivee2.pas';

var
  MaVariable: TClasseDeBase;
begin
  MaVariable := TClasseDeBase.Create;
  MaVariable.AfficherMessage;
  MaVariable.Free;

  MaVariable := TClasseDerivee1.Create;
  MaVariable.AfficherMessage;
  MaVariable.Free;

  MaVariable := TClasseDerivee2.Create;
  MaVariable.AfficherMessage;
  MaVariable.Free;

  Readln;
end.
```

Dans cet exemple, nous avons créé trois instances de classes différentes : une instance de la classe de base "TClasse de bas", une autre de la classe "TClasseDerivee1" et une dernière de classe "TClasseDerivee2".

Très important

L'héritage et le polymorphisme sont deux concepts fondamentaux de la programmation orientée objet en Delphi.

L'héritage consiste à créer une nouvelle classe à partir d'une classe existante en utilisant l'ensemble de ses propriétés et méthodes. La nouvelle classe hérite ainsi des caractéristiques de la classe parente, mais peut également ajouter de nouvelles fonctionnalités ou modifier celles de la classe parente. L'héritage permet de réutiliser le code existant et de créer des hiérarchies de classes.

Le polymorphisme, quant à lui, se réfère à la capacité d'un objet à prendre plusieurs formes. Cela signifie que les objets d'une classe peuvent être traités comme des objets d'une classe parente, mais agissent toujours en tant qu'objets de leur classe d'origine. Le polymorphisme permet de créer du code plus générique et flexible, car il permet de manipuler des objets de différentes classes à travers une interface commune.

L'héritage est le mécanisme permettant de créer des hiérarchies de classes, tandis que le polymorphisme permet de traiter les objets de différentes classes de manière homogène. Les deux concepts sont souvent utilisés ensemble pour créer des applications orientées objet efficaces et réutilisables.

7. Héritage multiple

L'héritage multiple en Delphi permet à une classe de dériver de plusieurs classes parentes. Cela peut être très utile pour réutiliser du code à partir de plusieurs classes existantes.

Voici un exemple pour illustrer l'utilisation de l'héritage multiple en Delphi :

Exemple 10 : Héritage multiple (Première classe parente)

```
unit MaClasseDeBase1;

interface

type
  TClasseDeBase1 = class
  public
    procedure AfficherMessage; virtual;
  end;

implementation

procedure TClasseDeBase1.AfficherMessage;
begin
  Writeln('Je suis une instance de TClasseDeBase1');
end;

end.
```

Exemple 11 : Héritage multiple (Deuxième classe parente)

```
unit MaClasseDeBase2;

interface

type
  TClasseDeBase2 = class
  public
    procedure AfficherMessage; virtual;
  end;

implementation

procedure TClasseDeBase2.AfficherMessage;
begin
  Writeln('Je suis une instance de TClasseDeBase2');
end;

end.
```

Dans cet exemple, nous avons défini deux classes de base différentes : "TClasseDeBase1" et "TClasseDeBase2". Chacune de ces classes contient une méthode publique appelée "AfficherMessage" avec le mot clé "virtual".

Maintenant, nous allons créer une classe qui dérive de ces deux classes parentes en utilisant l'héritage multiple :

Exemple 12 : Héritage multiple (Dérivation des classes précédentes)

```
unit MaClasseDerivee;

interface

uses
  MaClasseDeBase1, MaClasseDeBase2;

type
  TClasseDerivee = class(TClasseDeBase1, TClasseDeBase2)
  public
    procedure AfficherMessage; override;
  end;

implementation

procedure TClasseDerivee.AfficherMessage;
begin
  Writeln('Je suis une instance de TClasseDerivee');
end;

end.
```

Dans cet exemple, nous avons créé une classe dérivée appelée "TClasseDerivee" qui hérite des deux classes parentes "TClasseDeBase1" et

"TClasseDeBase2". Cette classe redéfinit la méthode "AfficherMessage" en utilisant le mot clé "override" pour indiquer qu'elle remplace les méthodes des classes parentes.

Maintenant, nous allons utiliser cette classe dans notre code principal :

Exemple 13 : Héritage multiple (Utilisation)

```
program MonProgramme;  
  
{$APPTYPE CONSOLE}  
  
uses  
  SysUtils, MaClasseDeBase1 in 'MaClasseDeBase1.pas',  
  MaClasseDeBase2 in 'MaClasseDeBase2.pas',  
  MaClasseDerivee in 'MaClasseDerivee.pas';  
  
var  
  MaVariable: TClasseDerivee;  
begin  
  MaVariable := TClasseDerivee.Create;  
  MaVariable.AfficherMessage;  
  MaVariable.Free;  
  
  Readln;  
end.
```

Dans cet exemple, nous avons créé une instance de la classe dérivée "TClasseDerivee" et appelé sa méthode "AfficherMessage". La sortie de ce programme sera "Je suis une instance de TClasseDerivee", car la méthode redéfinie dans cette classe remplace les méthodes des classes parentes.

Très important

L'héritage multiple en Delphi fait référence à la capacité d'une classe d'hériter de plusieurs autres classes. Cela permet à une classe d'avoir les propriétés et méthodes de plusieurs classes parentes.

Il est important de noter que l'héritage multiple peut conduire à des conflits de noms si les classes parentes ont des méthodes ou propriétés avec le même nom. Dans ce cas, il est nécessaire de spécifier explicitement quelle méthode ou propriété utiliser en préfixant le nom avec le nom de la classe parente appropriée.

8. Interfaces

Les interfaces en Delphi sont une autre méthode de polymorphisme. Elles permettent à une classe d'implémenter un comportement spécifique sans avoir à dériver d'une classe de base particulière. Les interfaces sont souvent utilisées pour fournir une implémentation standardisée de comportements communs dans plusieurs classes.

Voici un exemple pour illustrer l'utilisation des interfaces en Delphi :

Exemple 14 : Interface

```
unit MonInterface;

interface

type
    IMonInterface = interface
        procedure Methode1;
        function Methode2: Integer;
    end;

implementation

end.
```

Dans cet exemple, nous avons défini une interface appelée "IMonInterface" qui contient deux méthodes : "Methode1" qui ne renvoie pas de valeur, et "Methode2" qui renvoie un entier.

Maintenant, nous allons créer une classe qui implémente cette interface :

Exemple 15 : Classe implémentant l'interface

```
unit MaClasse;

interface

uses
    MonInterface;

type
    TMaClasse = class(TInterfacedObject, IMonInterface)
    public
        procedure Methode1;
        function Methode2: Integer;
    end;

implementation

procedure TMaClasse.Methode1;
begin
    Writeln('Je suis la méthode Methode1 de TMaClasse');
end;

function TMaClasse.Methode2: Integer;
begin
    Result := 42;
end;

end.
```

Dans cet exemple, nous avons créé une classe appelée "TMaClasse" qui implémente l'interface "IMonInterface". Cette classe utilise le mot clé

"TInterfacedObject" pour indiquer qu'elle implémente une interface, puis définit les deux méthodes de l'interface.

Maintenant, nous allons utiliser cette classe dans notre code principal :

Exemple 16 : Utilisation de la classe implémentant l'interface

```
program MonProgramme;  
  
{$APPTYPE CONSOLE}  
  
uses  
    SysUtils, MonInterface in 'MonInterface.pas', MaClasse in  
    'MaClasse.pas';  
  
var  
    MaVariable: IMonInterface;  
begin  
    MaVariable := TMaClasse.Create;  
    MaVariable.Methode1;  
    Writeln(MaVariable.Methode2);  
    MaVariable := nil;  
  
    Readln;  
end.
```

Dans cet exemple, nous avons créé une variable "MaVariable" de type "IMonInterface" et créé une instance de la classe "TMaClasse". Nous avons ensuite appelé les méthodes de l'interface à l'aide de cette variable. La sortie de ce programme sera "Je suis la méthode Methode1 de TMaClasse" et "42".

9. Cas pratique « gestion des documents »

Supposons que vous travaillez sur une application de gestion de bibliothèque. Vous avez des livres, des magazines et des journaux, chacun avec leurs propres attributs tels que le titre, l'auteur, la date de publication, le nombre de pages, etc.

Votre objectif est de créer une hiérarchie de classes pour représenter les différents types de documents, en utilisant l'héritage et la polymorphie. Vous devez également implémenter des méthodes pour ajouter et supprimer des documents, ainsi que pour afficher des informations sur chaque document.

Pour cela, vous pouvez créer :

- Une classe de base appelée "TDocument" qui contient les propriétés et les méthodes communes à tous les types de documents.
- Des classes dérivées telles que "TLivre", "TMagazine" et "TJournal" qui héritent de la classe "TDocument" et qui ont leurs propres propriétés et méthodes.

L'application doit permettre à l'utilisateur d'ajouter et de supprimer des documents, ainsi que d'afficher une liste de tous les documents avec leurs informations détaillées. Vous pouvez également ajouter des fonctionnalités supplémentaires, telles que la recherche de documents par titre ou auteur, ou la possibilité de trier les documents par date de publication.

Voici comment vous pouvez créer la classe de base TDocument en Delphi :

1. Ouvrez Delphi et créez un nouveau projet.
2. Dans le volet "Project Manager", cliquez avec le bouton droit de la souris sur le nom du projet et sélectionnez "New Unit".
3. Dans l'éditeur de code, tapez le code suivant pour définir la classe de base TDocument.

Exemple 17 : La classe TDocument

```
unit uDocument;

interface

type
  TDocument = class
  private
    FTitle: string;
    FAuthor: string;
    FPublicationDate: TDateTime;
    FPageCount: Integer;
  public
    constructor Create(const ATitle, AAuthor: string;
      APublicationDate: TDateTime; APageCount: Integer);
    procedure DisplayInfo;
    property Title: string read FTitle write FTitle;
    property Author: string read FAuthor write FAuthor;
    property PublicationDate: TDateTime read FPublicationDate
      write FPublicationDate;
    property PageCount: Integer read FPageCount write FPageCount;
  end;

implementation

constructor TDocument.Create(const ATitle, AAuthor: string;
  APublicationDate: TDateTime; APageCount: Integer);
begin
  FTitle := ATitle;
  FAuthor := AAuthor;
  FPublicationDate := APublicationDate;
  FPageCount := APageCount;
end;

procedure TDocument.DisplayInfo;
var
  Msg: string;
begin
  Msg := Format('Title: %s'#13#10'Author: %s'#13#10'Publication
    Date: %s'#13#10'Page Count: %d',
    [FTitle, FAuthor, DateToStr(FPublicationDate),
    FPageCount]);
  ShowMessage(Msg);
end;;

end.
```

Explications

- La classe TDocument est déclarée avec une section "private" et une section "public".
 - Dans la section "private", nous déclarons les champs privés qui représentent les propriétés du document : titre, auteur, date de publication et nombre de pages.
 - Dans la section "public", nous déclarons un constructeur pour initialiser les propriétés du document lorsqu'il est créé, une méthode DisplayInfo pour afficher les informations du document et des propriétés d'accès en lecture/écriture pour chaque propriété.
 - Le constructeur prend en paramètre les valeurs pour chaque propriété et initialise les champs privés correspondants.
 - La méthode DisplayInfo utilise la procédure Writeln pour afficher les informations du document.
4. Enregistrez le fichier en tant que "uDocument.pas".

Passons maintenant à la création de la classe dérivée TBook, qui hérite de la classe TDocument. Voici comment vous pouvez faire :

1. Créez une nouvelle unité (fichier) dans votre projet Delphi et nommez-la "uBook".
2. Dans l'éditeur de code, tapez le code suivant pour définir la classe dérivée TBook.

Exemple 18 : La classe TBook

```
unit uBook;

interface

uses
    uDocument;

type
    TBook = class(TDocument)
    private
        FISBN: string;
    public
        constructor Create(const ATitle, AAuthor, AISBN: string;
            APublicationDate: TDateTime; APageCount: Integer);
        procedure DisplayInfo; reintroduce;
        property ISBN: string read FISBN write FISBN;
    end;

implementation

constructor TBook.Create(const ATitle, AAuthor, AISBN: string;
    APublicationDate: TDateTime; APageCount: Integer);
begin
```

```

    inherited Create(ATitle, AAuthor, APublicationDate, APageCount);
    FISBN := AISBN;
end;

procedure TBook.DisplayInfo;
var
    Msg: string;
begin
    Msg := Format('Title: %s'#13#10'Author: %s'#13#10'Publication
Date: %s'#13#10'Page Count: %d'#13#10'ISBN: %s',
                [Title, Author, DateToStr(PublicationDate),
PageCount, ISBN]);
    ShowMessage(Msg);
end;

end.

```

Explications

- Nous utilisons l'unité `uDocument` avec la directive `uses` pour inclure la classe de base TDocument.
- Nous définissons une nouvelle classe TBook qui hérite de TDocument avec la directive `class(TDocument)`.
- Dans la section "private", nous ajoutons une nouvelle propriété FISBN qui représente l'ISBN du livre.
- Dans la section "public", nous déclarons un constructeur qui prend en paramètre les valeurs pour chaque propriété, et qui appelle le constructeur de la classe de base avec `inherited Create`.
- Nous redéfinissons la méthode `DisplayInfo` avec la directive `reintroduce`, pour afficher les informations du livre (en utilisant les propriétés de la classe de base et la propriété ISBN).
- Nous définissons une propriété d'accès en lecture/écriture pour la propriété ISBN.

1. Enregistrez le fichier en tant que "uBook.pas".

Et voilà, vous avez créé la classe dérivée TBook en Delphi, qui hérite de la classe de base TDocument.

Le prochain point concerne l'utilisation de la classe TBook dans votre application. Voici comment vous pouvez faire :

1. Dans votre formulaire principal (ou tout autre formulaire où vous souhaitez utiliser la classe TBook), ajoutez la directive `uses` pour inclure l'unité `uBook`.
2. Dans le code de votre formulaire, créez une nouvelle instance de la classe TBook en utilisant le constructeur que vous avez défini.

Exemple 19 : Utilisation de la classe TBook

```

var
  MyBook: TBook;
begin
  MyBook := TBook.Create('Tutorial Delphi', 'Just Soft',
    '1234567890', EncodeDate(2023, 5, 1), 30);
  try
    MyBook.DisplayInfo;
  finally
    MyBook.Free;
  end;

```

Explications

- Nous créons une nouvelle instance de la classe TBook avec le constructeur que nous avons défini, en passant les valeurs appropriées pour chaque propriété.
 - Nous utilisons la directive `try..finally` pour nous assurer que l'objet est libéré de la mémoire, même si une exception se produit.
3. Exécutez votre application et vous devriez voir une boîte de dialogue affichant les informations de votre livre, y compris son titre, son auteur, sa date de publication, son nombre de pages et son ISBN.

Et voilà, vous avez utilisé la classe TBook dans votre application Delphi. Vous pouvez maintenant utiliser cette classe pour gérer des livres dans votre application, en ajoutant des fonctionnalités supplémentaires comme la recherche, l'ajout ou la suppression de livres, etc.

Le prochain point que vous pouvez aborder concerne l'héritage et la création d'une sous-classe de TDocument, appelée TReport.

Voici comment vous pouvez procéder :

1. Créez une nouvelle unité appelée `uReport.pas`.
2. Dans cette unité, définissez une nouvelle classe appelée TReport, qui hérite de la classe TDocument.

Exemple 20 : Création de la classe TReport

```

type
  TReport = class(TDocument)
  private
    FReportDate: TDateTime;
  public
    constructor Create(const ATitle, AAuthor, AISBN: string;
      APublishedDate: TDateTime; APageCount: Integer; AReportDate:
      TDateTime);
    procedure DisplayInfo; override;
  end;

```


Explications

- Nous définissons une nouvelle classe TReport, qui hérite de la classe TDocument avec la syntaxe `class TReport = class(TDocument)`.
- Nous déclarons une nouvelle propriété privée appelée FReportDate pour stocker la date de rapport spécifique à TReport.
- Nous définissons un nouveau constructeur appelé Create, qui prend les mêmes paramètres que le constructeur de TDocument, ainsi qu'un paramètre supplémentaire pour la date de rapport.
- Nous définissons également une nouvelle méthode DisplayInfo, qui utilise `inherited` pour appeler la méthode DisplayInfo de la classe de base TDocument, et affiche ensuite la date de rapport spécifique à TReport.

3. Implémentez le constructeur et la méthode DisplayInfo de TReport.

Exemple 21 : Implémentation de la classe TReport

```

constructor TReport.Create(const ATitle, AAuthor, AISBN: string;
APublishedDate: TDateTime; APageCount: Integer; AReportDate:
TDateTime);
begin
    inherited Create(ATitle, AAuthor, AISBN, APublishedDate,
APageCount);
    FReportDate := AReportDate;
end;

procedure TReport.DisplayInfo;
begin
    inherited DisplayInfo;
    ShowMessage(Format('Date du rapport: %s',
[DateToStr(FReportDate)]));
end;

```

Explications

- Dans le constructeur de TReport, nous appelons d'abord le constructeur de la classe de base TDocument en utilisant `inherited Create`, puis nous stockons la date de rapport spécifiée dans la propriété FReportDate.
 - Dans la méthode DisplayInfo de TReport, nous appelons d'abord la méthode DisplayInfo de la classe de base TDocument en utilisant `inherited DisplayInfo`, puis nous affichons la date de rapport spécifique à TReport à l'aide de `ShowMessage` et `Format`.
4. Dans votre formulaire principal (ou tout autre formulaire où vous souhaitez utiliser la classe TReport), créez une nouvelle instance de la classe TReport en utilisant le constructeur que vous avez défini.

Exemple 22 : Utilisation de la classe TReport

```
var
```

```
MyReport: TReport;  
begin  
  MyReport := TReport.Create('Utilisation de la POO', 'Just Soft',  
    '1234567890', EncodeDate(2023, 5, 1), 10, EncodeDate(2023, 5, 1));  
  try  
    MyReport.DisplayInfo;  
  finally  
    MyReport.Free;  
  end;
```

Explications

- Nous créons une nouvelle instance de la classe TReport avec le constructeur que nous avons défini, en passant les valeurs appropriées pour chaque propriété, y compris la date de rapport spécifique à TReport.
 - Nous utilisons la directive `try..finally` pour nous assurer que l'
5. Exécutez votre application pour voir les informations affichées à l'écran. Vous devriez voir le titre, l'auteur, l'ISBN, la date de publication, le nombre de pages et la date de rapport pour votre instance de TReport affichés à l'écran.

En créant cette sous-classe de TDocument, vous avez pu réutiliser une grande partie du code que vous aviez déjà écrit pour la classe de base. Cela montre comment l'héritage peut être utilisé pour réduire la duplication de code et simplifier le développement.

Cependant, il est important de noter que l'héritage peut également entraîner une complexité supplémentaire, notamment en ce qui concerne la gestion de l'héritage multiple et la conception de hiérarchies de classes appropriées. Il est donc important de bien comprendre les concepts d'héritage et de planifier la hiérarchie de classes avec soin avant de commencer à écrire du code.

Enfin, pour continuer à améliorer votre exemple, vous pouvez envisager d'ajouter d'autres sous-classes de TDocument pour représenter différents types de documents, tels que TBook, TArticle, etc. Chaque sous-classe peut ajouter des propriétés et des méthodes spécifiques à ce type de document, tout en héritant des propriétés et des méthodes communes de la classe de base TDocument.